

Expt 4: Hamming code:

Aim

To write and implement a program for Hamming Code generation to detect and correct a single-bit error in the transmitted data.

Theory

Hamming Code is an error-detecting and error-correcting technique used in digital communication and computer networks. It adds a number of redundant (parity) bits to the original data bits so that errors introduced during transmission can be detected and corrected at the receiver.

The number of parity bits required is determined using:

$$2^r \geq m + r + 1$$

Part 1: Hamming Code Generation Algorithm

Input: Binary data bits (e.g., 1011)

Output: Hamming Code with redundant parity bits

Algorithm Steps:

1. Start
2. Read the binary data string data_bits
3. Let m = length of data_bits
4. Initialize r = 0
5. Find the number of parity bits (r) such that $2^r \geq m + r + 1$
6. Let n = m + r (total length of Hamming code)
7. Create an array hamming_code[1 to n]
8. Place data bits in positions not equal to 2^i (i.e., 1, 2, 4,... are reserved for parity bits)
9. For each parity bit position p = 2^i :
 - a. Initialize parity = 0
 - b. For every position j from 1 to n:
 - If j & p != 0, include hamming_code[j] in parity calculation
 - c. Set hamming_code[p] = parity % 2 (XOR result)
10. Print hamming_code
11. End

Part 2: Error Detection and Correction Algorithm

Input: Received Hamming code

Output: Position of error (if any), and corrected code

Algorithm Steps:

1. Start
2. Read the received hamming code as array `received_code[1 to n]`
3. Initialize `error_position = 0`
4. For each parity position $p = 2^i$:
 - a. Initialize `parity = 0`
 - b. For every position `j` from 1 to `n`:
 - If $j \& p \neq 0$, include `received_code[j]` in parity check
 - c. If `parity  $\neq$  0`, then add `p` to `error_position`
5. If `error_position == 0`:
 - No error
6. Else:
 - Flip the bit at `error_position` in `received_code` (i.e., correct the bit)
7. Print corrected code
8. End

Notes:

- `&` is a bitwise AND operator
- 2^i parity positions: 1, 2, 4, 8,...
- XOR is used to calculate parity

Python Program:

```
def calculate_parity_bits(data_bits):  
    # Length of the data bits  
    m = len(data_bits)  
    r = 0  
    # Find the number of redundant bits needed (r)  
    while (2 ** r) < (m + r + 1):  
        r += 1  
    # Total length of Hamming code
```

```

n = m + r

# Initialize the code array with placeholders (indexing from 1)
hamming_code = ['x'] * (n + 1)

# Place data bits in their positions (non 2^i positions)
j = 0
for i in range(1, n + 1):
    if (i & (i - 1)) != 0: # Not a power of 2
        hamming_code[i] = int(data_bits[j])
        j += 1

# Calculate parity bits
for i in range(r):
    parity_pos = 2 ** i
    parity = 0
    for k in range(1, n + 1):
        if k & parity_pos and k != parity_pos:
            if hamming_code[k] != 'x':
                parity ^= hamming_code[k]
    hamming_code[parity_pos] = parity

return hamming_code[1:] # Remove the dummy 0-th index

def introduce_error(code, position):
    # Flip the bit at the given position to introduce an error
    if 1 <= position <= len(code):
        code[position - 1] ^= 1
    return code

def detect_error(received_code):
    n = len(received_code)
    r = 0
    while (2 ** r) < (n + 1):
        r += 1
    error_position = 0

```

```

for i in range(r):
    parity_pos = 2 ** i
    parity = 0
    for k in range(1, n + 1):
        if k & parity_pos:
            parity ^= received_code[k - 1]
    if parity != 0:
        error_position += parity_pos
return error_position

```

```

# --- MAIN PROGRAM ---

```

```

# Input binary data (e.g., '1011')

```

```

data = input("Enter data bits (e.g., 1011): ")

```

```

# Generate Hamming code

```

```

hamming = calculate_parity_bits(data)

```

```

print("Generated Hamming Code:", ".join(map(str, hamming)))

```

```

# Introduce error at a position (optional)

```

```

error_pos = int(input("Enter position to introduce error (0 for none): "))

```

```

if error_pos != 0:

```

```

    hamming = introduce_error(hamming, error_pos)

```

```

    print("Hamming Code with error introduced:", ".join(map(str, hamming)))

```

```

# Detect error

```

```

detected_pos = detect_error(hamming)

```

```

if detected_pos == 0:

```

```

    print("No error detected.")

```

```

else:

```

```

    print(f"Error detected at position: {detected_pos}")

```

Output:

1. Enter data bits (e.g., 1011): **1101**

Enter position to introduce error (0 for none): **0**

Generated Hamming Code: 1010101

No error detected.

2. Enter data bits (e.g., 1011): **1101**

Enter position to introduce error (0 for none): **2**

Generated Hamming Code: 1010101

Hamming Code with error introduced: 1110101

Error detected at position: 2

Applications

1. Digital communication systems
2. Radio communication
3. Computer networks
4. Optical communication
5. Error-control coding systems